

ANHANGUERA-LONDRINA/PR

ENGENHARIA DE SOFTWARE

PORTFÓLIO - RELATÓRIO DE AULA PRÁTICA:

NOME DA DISCIPLINA: FUNDAMENTOS DE INTELIGÊNCIA ARTIFICIAL

Londrina/PR
2025

PORTFÓLIO – RELATÓRIO DE AULA PRÁTICA

NOME DA DISCIPLINA: FUNDAMENTOS DE INTELIGÊNCIA ARTIFICIAL

Trabalho de portfólio apresentado como requisito parcial para a obtenção de pontos para a média semestral.

Orientador: Tutor

SUMÁRIO

1. INTRODUÇÃO..... 4
2. DESENVOLVIMENTO..... 4
3. CONCLUSÃO..... 7

INTRODUÇÃO

A atividade prática aplica conceitos fundamentais de Inteligência Artificial em duas seções: a primeira utiliza a lógica nebulosa para modelar decisões subjetivas, como determinar a gorjeta de um restaurante com base na qualidade da comida e do serviço; a segunda foca na implementação de redes neurais artificiais em Python, demonstrando como modelos de aprendizado supervisionado são treinados para reconhecer padrões e realizar previsões a partir de dados históricos.

2. DESENVOLVIMENTO

2.1 LÓGICA NEBULOSA EM SISTEMA ESPECIALISTA

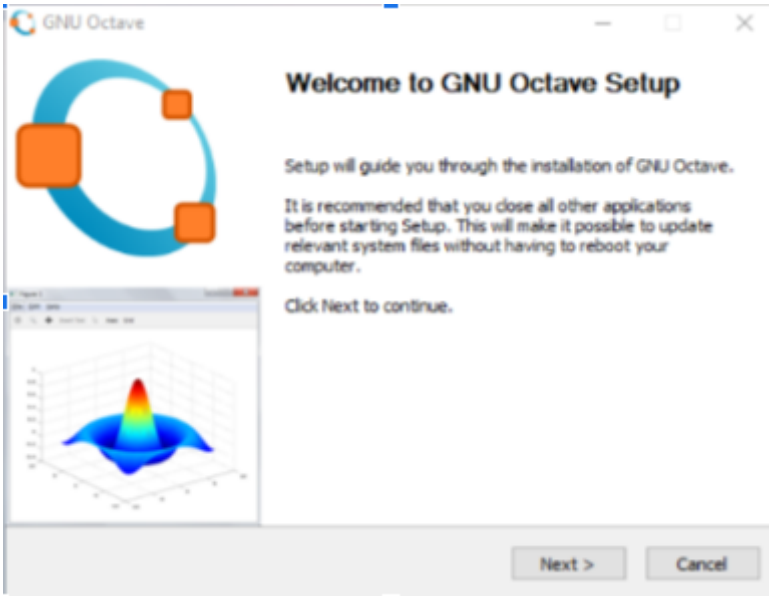
Checklist da Atividade Prática:

- Octave está funcionando corretamente.
- Criar as variáveis necessárias.
- Inserir as regras corretamente.
- Gerar valores para a gorjeta com base em diferentes qualidades de serviço e comida.
- Registrar seus resultados em um relatório.

A seguir, vamos realizar a avaliação de um restaurante usando uma abordagem procedural dentro do octave. Para realizar esse cálculo da gorjeta foram utilizados dois parâmetros principais, qualidade da comida e qualidade do serviço. Ambos os parâmetros estão em formato de nota que possui uma escala de 0 a 10. Ao final da função, o programa vai determinar o valor da gorjeta com base nesses dois parâmetros passados para a função de média.

A primeira etapa do trabalho é instalar e configurar o octave, ele está disponível em formato .exe para o sistema operacional Windows, depois de baixar o executável o usuário tem que seguir um passo a passo básico com o intuito de realizar a configuração inicial do programa dentro do ambiente operacional.

Figura 1. Instalação do octave



Para mostrar como a lógica nebulosa pode ser usada nesse problema, um novo código foi desenvolvido para realizar esse cálculo de maneira mais agrupada de acordo com a avaliação de cada critério avaliado pelo cliente.

O primeiro passo foi definir o sistema no octave e inserir os dados de entrada que serão usados no cálculo do sistema nebuloso. As variáveis escolhidas foram a nota da comida e a nota do serviço, juntos eles vão definir a gorjeta final que o restaurante irá receber.

Figura 2. Definição do sistema de lógica nebulosa (fuzzy logic) e as variáveis de entrada do sistema

restaurante_teste.m

```
1  pkg load fuzzy-logic-toolkit
2
3  % Definindo o sistema de lógica nebulosa
4  fis = newfis('calculadora_gorjeta');
5
6  % Adicionando variáveis de entrada (comida e serviço)
7  fis = addvar(fis, 'input', 'comida', [0 10]);
8  fis = addvar(fis, 'input', 'servico', [0 10]);
9
10 % Funções de pertinência para comida
11 fis = addmf(fis, 'input', 1, 'Ruim', 'trapmf', [-1 0 3 5]);
12 fis = addmf(fis, 'input', 1, 'Mediano', 'trapmf', [3 5 6 8]);
13 fis = addmf(fis, 'input', 1, 'Bom', 'trapmf', [6 8 10 11]);
14
15 % Funções de pertinência para serviço
16 fis = addmf(fis, 'input', 2, 'Ruim', 'trapmf', [-1 0 3 5]);
17 fis = addmf(fis, 'input', 2, 'Mediano', 'trapmf', [3 5 6 8]);
18 fis = addmf(fis, 'input', 2, 'Bom', 'trapmf', [6 8 10 11]);
```

Figura 3. Definição da variável de saída e as regras da lógica nebulosa

```
% Adicionando variável de saída (gorjeta)
fis = addvar(fis, 'output', 'gorjeta', [5 15]);

% Funções de pertinência para gorjeta
fis = addmf(fis, 'output', 1, 'Baixa', 'trapmf', [4 5 7 9]);
fis = addmf(fis, 'output', 1, 'Media', 'trapmf', [7 9 11 13]);
fis = addmf(fis, 'output', 1, 'Alta', 'trapmf', [11 13 15 16]);

% Adicionando regras
ruleList = [
    1 1 1 1 1;
    2 2 2 1 1;
    3 3 3 1 1;
    1 2 2 1 1;
    2 1 2 1 1;
    1 3 2 1 1;
    3 1 2 1 1;
    2 3 3 1 1;
    3 2 3 1 1;
];
fis = addrule(fis, ruleList);
```

Por último, o código recebe as notas de comida e serviço e gera uma porcentagem de gorjeta para o cliente pagar ao restaurante.

Figura 4. Código para o cálculo da gorjeta

```

comida = 7
servico = 4
input = [comida servico];
gorjeta = evalfis(input, fis);
fprintf('Comida: %d, Serviço: %d -> Gorjeta sugerida: %.2f%%\n', comida, servico, gorjeta);
```

2.2 ALGORÍTMOS DE REDES NEURAIS

Checklist da Atividade Prática:

- Entender o conceito de redes neurais artificiais;
- Implementar através do Python uma RNA simples.
- Realizar e entender os resultados da quantidade de treinamentos.

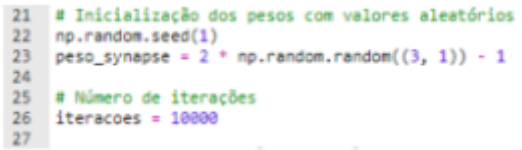
O primeiro passo da atividade é fazer a importação do módulo numpy para auxílio dos cálculos da rede neural proposta. A plataforma jdoodle foi usada para o desenvolvimento do projeto.

Em seguida, devemos fazer a definição do sigmóide e sua derivada que será usado na rede neural. A definição de um conjunto de dados de entrada e saída também é de extrema importância para ajudar na determinação de padrões pelo algoritmo de classificação da rede neural, os dados de entrada são chamados de x e os dados de saída são chamados de y dentro do código.

Os pesos da sinapse serão inicializados com valores aleatórios. É comum inicializar os pesos com valores pequenos e aleatórios para quebrar a simetria e permitir o

aprendizado. O código abaixo ilustra exatamente a definição desse peso e o número de iterações do algoritmo da rede neural:

Figura 5. Definição do peso e do número de iterações



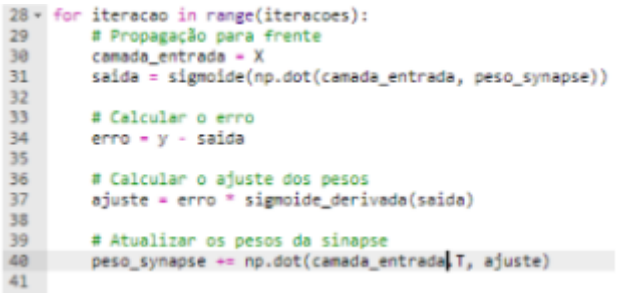
```
# Inicialização dos pesos com valores aleatórios
np.random.seed(1)
peso_synapse = 2 * np.random.random((3, 1)) - 1

# Número de iterações
iteracoes = 10000

```

Em seguida, o algoritmo entra em um loop de repetição para realizar o ajuste da rede neural usando o sigmóide e sua derivada, também é de extrema importância realizar o ajuste na rede usando os dados de saída para obter o melhor desempenho do algoritmo de acordo com os dados de entrada.

Figura 6. Loop de treinamento da rede neural



```
for iteracao in range(iteracoes):
    # Propagação para frente
    camada_entrada = X
    saida = sigmoide(np.dot(camada_entrada, peso_synapse))

    # Calcular o erro
    erro = y - saida

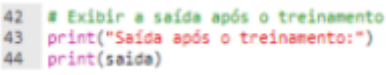
    # Calcular o ajuste dos pesos
    ajuste = erro * sigmoide_derivada(saida)

    # Atualizar os pesos da sinapse
    peso_synapse += np.dot(camada_entrada.T, ajuste)

```

Por último, os dados de saída da rede são mostrados para o usuário para demonstrar a eficiência do algoritmo para o conjunto de dados fornecidos para o treinamento da rede neural.

Figura 7. Código para imprimir a saída da rede neural



```
# Exibir a saída após o treinamento
print("Saída após o treinamento:")
print(saida)
```

CONCLUSÃO

Esta atividade prática demonstrou com sucesso a aplicação de duas abordagens centrais da Inteligência Artificial: a lógica nebulosa, que se mostrou eficaz em modelar a subjetividade e a incerteza de avaliações humanas (como no cálculo de gorjetas), e as redes neurais artificiais, que ilustraram a capacidade do aprendizado supervisionado em reconhecer padrões e fazer previsões a partir de dados históricos.